Ders 10 Genişletilmiş Notu: Ara Sınav Sorusu Üzerine Tartışma: Prototype ve Abstract Factory

Ders 10 Genişletilmiş Notu: Ara Sınav Sorusu Üzerine Tartışma: Prototype ve Abstract Factory

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bu Kaydın Kapsamı
- 2. Sınav Sorusu: Çok Sayıda Benzer Nesne ve Az Bellek
- 3. Factory Tabanlı Yanıtlar ve Abstract Factory'nin Kırılım Sorunu
- Örnek Sorular
- Dersin Toparlaması

1. Bu Kaydın Kapsamı

Bu hafta yeni konu işlenmedi. Hocanın ekrandaki duyurusu, bugün ders işlenmeyeceğini ve soruları olanların mikrofon açıp sorabileceğini söyler (ekrandan, 00:07:40). Oturum bir süre soru için açık tutuldu. Kaydın akademik kısmı, birinci ara sınavın ikinci sorusu üzerine bir öğrencinin sorusuyla başlayan birkaç dakikalık tartışmadır (00:21:40 - 00:25:55).

Bir önceki mevcut kayıtta (ders 9) tasarım kalıpları Mediator ile bitirilmiş ve refactoring konusuna geçilmişti. Bu kayıttaki tartışma ise daha önce işlenen yaratımsal (creational) kalıplara, Prototype ve Abstract Factory'ye geri döner.

Ekranda bu sırada yalnız duyuru slaydı ve hocanın dosya gezgini görünür (ekrandan, 00:23:10). Sınav sorusunun metni gösterilmedi; hoca soruyu tümüyle paylaşmak istemediğini söyledi. Bu yüzden aşağıdaki soru tanımı yalnız hocanın sözlü anlatımına dayanır.

2. Sınav Sorusu: Çok Sayıda Benzer Nesne ve Az Bellek

Hocanın anlattığına göre ikinci soruda bir dolu ders vardır. Bu derslerin birbirine çok benzeyen pek çok ortak özelliği vardır; aralarında yalnız ufak tefek değişiklikler bulunur. Ders sayısı çok olduğu için birçok yerde yüksek bellek kullanımı ortaya çıkabilir. Soru bu nedenle bellek kullanımının mümkün olduğu kadar az tutulmasını açıkça ister.

Hoca, bu kadar net bir gereksinim bulunduğu için sorunun tek doğru yanıtının Prototype olduğunu söyledi. Buradaki ilke, kalıbın gereksinimden seçilmesidir: soruda "çok sayıda benzer nesne" ve "az bellek" birlikte geçiyorsa, aranan kalıp Prototype'tır.

> **Hocanın vurgusu:** Böyle net bir gereksinim olduğu için o soruda tek yanıt Prototype'tır.

> **Ek bilgi (derste anlatılmadı):** Prototype kalıbında yeni nesne, sınıfından sıfırdan kurulmak yerine hazır bir örnek nesnenin kopyalanmasıyla (clone) elde edilir. Ortak özellikler prototip nesnede bir kez hazırlanır, kopyada yalnız farklı olan alanlar değiştirilir. Bu, büyük olasılıkla hocanın soruda aradığı "ortak kısımları bir kez tut, farkları sonradan ver" yaklaşımıdır.

3. Factory Tabanlı Yanıtlar ve Abstract Factory'nin Kırılım Sorunu

3.1 Öğrencinin önerisi

Soru: İkinci soruyu factory şeklinde yapsak olmaz mı? Örneğin factory'yi bir depo (store) gibi kullanıp nesneleri hem orada oluştursak hem de orada biriktirsek?

Cevap: Hoca, factory tabanlı yanıtların hepsinin çeşitli bellek sıkıntılarına yol açtığını söyledi. Abstract Factory belki olabilir, ama onda da kalıtım ilişkisinin kırılımı açısından çok önemli sorunlar vardır.

3.2 Abstract Factory bu bağlamda nasıl kurulurdu?

Hoca, Abstract Factory'nin hangi tür bir duruma uyduğunu bir örnekle anlattı. Abstract Factory'de bir ölçüte göre ürün aileleri oluşur. Bölümde birden fazla program olduğunu düşünelim: %100 Türkçe, %30 İngilizce ve %100 İngilizce. Bu programların her biri bir ürün ailesidir.

Programların dersleri de bu ailelerin bireyleridir. Örneğin Nesneye Yönelik Programlama dersi:

- %100 Türkçe programda Türkçe verilir.
- %100 İngilizce programda İngilizce verilir.
- %30 İngilizce programda duruma göre Türkçe ya da İngilizce açılabilir.

Bunun yanında derslerin zorunlu ya da seçimlik olması gibi bir ayrım da vardır. Bu yüzden asıl soru, ürün ailesini ve bu ailenin bireylerini hangi aileye göre yazacağınızdır. Kalıtım kırılımı doğru kurulmazsa çözüm Abstract Factory'nin amacına uymaz.

3.3 İki düzey kalıtım

Hoca son olarak Abstract Factory kalıbının yapısında iki düzey kalıtım bulunduğunu hatırlattı. Bazı yanıtlar soruyu tek düzeyli bir kalıtım ağacıyla çözmeye çalışmıştı; hocaya göre bu, kalıbın UML gösterimine uymaz.

> **Ek bilgi (derste anlatılmadı):** Abstract Factory'nin standart yapısında iki ayrı kalıtım hiyerarşisi bulunur: soyut fabrikadan türeyen somut fabrikalar (her ürün ailesi için bir tane) ve her ürün türü için soyut üründen türeyen somut ürünler. Bir somut fabrika, kendi ailesine ait somut ürünleri oluşturur. Hocanın "iki düzey" sözü büyük olasılıkla bu yapıya işaret eder.

Örnek Sorular

Soru 1 (derste sözlü, 00:22:41): Çok sayıda ders vardır; bu derslerin birbirine çok benzeyen ortak özellikleri vardır ve aralarında yalnız küçük farklar bulunur. Ders sayısı fazla olduğu için bellek kullanımının mümkün olduğunca az tutulması istenmektedir. Bu gereksinimi karşılayan tasarım kalıbı hangisidir?

> **Çözüm (derste verildi):** Prototype. Gereksinim çok sayıda benzer nesnenin az bellekle oluşturulmasıdır. Factory tabanlı çözümler bellek sorunlarına yol açar; Abstract Factory'de ise ürün ailesi kırılımı ve iki düzey kalıtım açısından sorunlar vardır.

Soru 2 (derste sözlü, 00:24:10): %100 Türkçe, %30 İngilizce ve %100 İngilizce programları ile bu programlarda verilen dersler (örneğin Nesneye Yönelik Programlama) Abstract Factory ile modellenecek olsa ürün aileleri ve ailelerin bireyleri ne olurdu?

> **Çözüm (derste verildi):** Ürün aileleri programlardır (%100 Türkçe, %30 İngilizce, %100 İngilizce); bireyler bu programlarda verilen derslerdir. Aynı ders %100 Türkçe programda Türkçe, %100 İngilizce programda İngilizce, %30 İngilizce programda duruma göre Türkçe ya da İngilizce olur. Zorunlu ve seçimlik ayrımı da göz önüne alınarak kırılım doğru kurulmalı ve kalıbın iki düzey kalıtımı gösterilmelidir.

Dersin Toparlaması

Bu kayıtta yeni konu işlenmedi; akademik içerik, birinci ara sınavın ikinci sorusuna dair kısa bir tartışmadır. Tartışmanın ana dersi, kalıp seçiminin sorudaki gereksinimden yapılmasıdır: çok sayıda, birbirine çok benzeyen nesne ve az bellek gereksinimi birlikte Prototype kalıbını gösterir. Factory tabanlı çözümler bu soruda bellek sorunlarına yol açar. Abstract Factory ise ürün ailelerinin (örnekte programların) ve bu ailelerin bireylerinin (derslerin) doğru kırılımla ayrıldığı, iki düzey kalıtım içeren bir yapıdır; tek düzeyli bir kalıtım ağacı bu kalıba uymaz.

Bir önceki mevcut kayıtta (ders 9) başlanan refactoring konusu bu kayıtta ilerlemedi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.